Getting Started with a CESR Workstation

Standard Procedure

1. Confirm you have a CESR laptop or device to use.

   These devices have a sticker that says "COCESR-XXXX," where XXXX is a number.

2. Confirm that you have been approved to use the workstation you intend to access.

   Your principal investigator (PI) or project manager should have approved your request. Ask them if you are unsure which workstation or account you may use.

3. Check that your CESR computer is connected to the internet.

4. Make sure Visual Studio Code (`VS Code`) is installed on your computer and open it from the applications menu.

   If it is missing, contact your PI, project manager, or the Mechanical Engineering IT group, as `VS Code` is approved CESR software.

Install the Remote SSH extension

In `VS Code`, select the Extensions icon on the left sidebar. Enter `SSH` in the extension search field. Install `Remote - SSH`, published by Microsoft. This extension lets `VS Code` connect to the workstation through Secure Shell (SSH). If `VS Code` asks you to reload after installation, do so.

Add the workstation

Open the Command Palette with `Ctrl+Shift+P`. Type `Remote-SSH: Add New SSH Host` and select that command. When `VS Code` asks for the SSH connection command, enter it in the following form, replacing the placeholders. The `-p` option selects the SSH port. The `-l` option supplies your login name (lowercase L).

```
ssh -p PORT_NUMBER -l USERNAME@cesr.sc.edu COMPUTER_IP_ADDRESS
```

For example, I would enter the following command to connect to `COCESR-1141`.

```
ssh -p 222 -l jvaught@cesr.sc.edu 129.252.23.212
```

After entering the command, select the SSH configuration file in which `VS Code` should save the connection. Choose the first, default option for your own user account, unless you know what you are doing and want to select a different config file. This saves the connection details so that you can select the workstation next time.

Connect to the workstation

Select Remote Explorer on the left sidebar. If necessary, select the SSH view in its drop-down. Find the workstation you just added and move the pointer over its entry to show the connection buttons.

The arrow connects in the current window. The window-shaped button with a plus opens a new window. Either option works for this guide, however I will use the arrow to continue in the current window. You can also use `Remote-SSH: Connect to Host` from the Command Palette if you do not want to use the mouse.

On the first connection, `VS Code` may ask which operating system the remote workstation uses. Select `Linux`.

SSH may also ask you to confirm the workstation's host key. This identifies the machine you are connecting to. You may be asked to re-confirm this if the SSH configurations change on the workstation.

When prompted, enter your CESR account password, the password you use for your CESR computer and email. Your password characters will be hidden. Wait for `VS Code` to finish connecting and installing its remote components. The remote indicator in the lower-left corner should identify the connected workstation.

IMPORTANT: If you forget your password

If you have forgotten your password, and want to change it, sign in at https://myaccount.microsoft.com/ and select `Change Password`. Follow the prompts for your work account.

Folder Access

After connecting, select `Open Folder` in the Explorer sidebar. This will give the editor a working location and lets you manage files in Explorer. You can create, rename, move, and delete files with the mouse, and transfer files from your computer by dragging them into the remote Explorer. **NOTE:** You do not need to open a folder to use the terminal.

I recommend choosing your home folder. Its folder name is your account name, and it is usually the default location offered for a new connection. If you choose, you can instead open another folder you are authorized to read and write (including the HDD).

`VS Code` may ask whether you trust the authors of the files in the folder. If you created the files or trust their source, accept the workspace-trust prompt. If you believe files in your home folder are unsafe or unexpected, alert your PI or project manager.

Terminal Access

You can open a terminal through the Terminal menu, then New Terminal. The shortcut `Ctrl+‘` also opens the terminal (note: the small tick is the backtick key, shared with the tilde symbol `~` on your keyboard). The terminal in the connected `VS Code` window will now run on the remote workstation.

To check your access, type the following command and press Enter.

```
hostname
```

As an example, if you connected to `COCESR-1141`, the output should be as follows.

```
cocesr-1141
```

A prompt normally returns when a command finishes, however some successful commands display no output and return you to the terminal prompt. In many Windows and Linux terminal applications, `Ctrl+Shift+C` copies selected text and `Ctrl+Shift+V` pastes it. **IMPORTANT:** The command `Ctrl+C` STOPS a running terminal program.

Test the Compute Environment

First, create a folder for the exercise. The `mkdir` command creates(or 'makes') directories. The `-p` option creates missing parent folders and allows the folder to exist already. The `~` symbol means your home folder.

```
mkdir -p ~/Documents/hello-practice
```

Next, enter (or 'open') that folder with `cd`, which means change directory.

```
cd ~/Documents/hello-practice
```

Download the example with `curl`. The `-fail` option reports HTTP errors, `-location` follows redirects, and `-output` names the saved file. Copy both lines together; the backslash continues the command on the next line.

```
curl --fail --location --output hello.py \
"https://raw.githubusercontent.com/j-vaught/server-guide/main/hello.py"
```

If the download reports an error, resolve it before continuing. Then check that Python is available. This command prints the installed Python version.

```
python3 --version
```

Finally, run the downloaded file with Python.

```
python3 hello.py
```

The program should print a greeting, the workstation name, and the Python version. If `python3` is unavailable, ask for assistance from the PI/Project Manager or IT.

1 Installing Additional Software Tools

Once you have gotten connected and have the simple demo Python program working, you may decide you want more features — more control, better validation, type checking, etc. Ergo, I am listing the install process for some key software components I've seen used on larger projects.

1.1 Git and GitHub

The absolute most common additional piece of software people will ask for — beyond an operating system, drivers, and python — is a Source Control or Version Control system. Today, there are naturally a whole slew of options to pick from, but realistically, everyone uses `Git`. Here, I will walk through the installation steps and some basic usage.

Installing Git

Run the following commands in the remote Linux terminal. First, make programs installed under your account available to this terminal. This changes where the shell looks for commands.

```
export PATH="$HOME/.local/bin:$PATH"
```

Check whether `Git` is already available.

```
git --version
```

If a version appears, skip the download and installation below. Otherwise, enter your Documents folder.

```
cd ~/Documents
```

Download the installer from this guide's repository.

```
curl --fail --location --output git.sh \
  https://raw.githubusercontent.com/j-vaught/server-guide/main/git.sh
```

Run the downloaded installer. It installs `Git` and its supporting libraries under your account for you to access only; this is NOT global installation.

```
sh git.sh
```

After installation finishes, confirm that `Git` runs.

```
git --version
```

The `export` command applies to the current terminal. Run it again in a new terminal, or add that same line to your `~/.bashrc` file to apply it automatically in future Bash terminals.

Create a repository and record a change

Create a new practice folder. Use a new folder for this exercise so that you do not change an existing project.

```
mkdir -p ~/Documents/git-practice
```

Enter the practice folder.

```
cd ~/Documents/git-practice
```

Initialize a repository in that folder.

```
git init
```

Create its initial branch named `main`. A branch is a named line of development.

```
git checkout -b main
```

Set the author name attached to commits in this repository. Replace the placeholder with your own name.

```
git config user.name "YOUR NAME"
```

Set the author's email address, replacing the placeholder with your own email.

```
git config user.email "YOUR_EMAIL"
```

In `VS Code`, create a file named `README.txt` inside the practice folder. Write a sentence about the project and save the file. Return to the remote terminal and inspect the repository.

```
git status
```

The new file should appear as untracked, meaning it has not yet been included in a commit. Select its current contents for the next commit with `git add`. This selection is called staging.

```
git add README.txt
```

Record the staged contents with `git commit`. The `-m` option supplies a message describing the change.

```
git commit -m "Add project notes"
```

Inspect the recorded history. The short string before the message identifies the commit.

```
git log --oneline
```

To practice another change, edit and save `README.txt` again. Inspect the difference before recording it.

```
git diff
```

Repeat `git add README.txt` and `git commit` with a new message to record the update.

1.2 Python environments and packages

Different projects often require different Python versions and different versions of the same package. One way to keep your head straight about all of these version conflicts is to use an environment, which keeps these dependencies separate from other projects.

While there are a few environment managers, the one I use the most often is `uv`. `uv` manages Python versions, project environments, and packages. It also records dependencies so that it is easy to recreate the project's environment.

A more often used manager in scientific work is `Conda` and `Mamba`, both of which manage environments containing Python packages and other software, including compiled libraries and command-line tools. Below, we will install both through `Miniforge`, which uses the `conda-forge` package collection by default.

Last, and least of all, Python also includes `venv`, which creates an environment from an existing Python installation. It is commonly paired with `pip` to install packages.

Installing uv

To install uv, run the following command in the remote Linux terminal.

```
curl -LsSf https://astral.sh/uv/install.sh | sh
```

After installation, open a new remote terminal so that it recognizes the new command. Check the installation.

```
uv --version
```

Using uv

With `uv` installed, enter your Documents folder.

```
cd ~/Documents
```

Create a new project named `UV-demo` that uses Python 3.11. If a suitable Python installation is unavailable, `uv` will automatically download it.

```
uv init --python 3.11 UV-demo
```

Enter the project folder.

```
cd UV-demo
```

Install a package with `uv add`. For example, the following command installs NumPy version 1.26.4.

```
uv add "numpy==1.26.4"
```

Use `uv run` to run a command inside the project's environment. For example, check its Python version.

```
uv run python --version
```

Check the installed NumPy version.

```
uv run python -c "import numpy; print(numpy.__version__)"
```

To run your own Python file, put `uv run` before its normal command. For example, if the current folder contains `hello.py`, run `uv run python hello.py`. One thing to keep in mind is to keep `pyproject.toml` and `uv.lock` with your project in Git.

After downloading an existing uv project, you can download all dependencies via.

```
uv sync --locked
```

The `-locked` option installs the package versions recorded in `uv.lock`, adding missing packages or replacing incorrect versions in the project's environment. If the lockfile is missing or no longer matches the requirements in `pyproject.toml`, the command stops with an error instead of updating the lockfile.

Installing Miniforge

If you prefer `Conda` or `Mamba`, install `Miniforge` with the following command in the remote Linux terminal.

```
curl -fsSL \
https://raw.githubusercontent.com/j-vaught/server-guide/main/miniforge.sh
   | sh
```

The script installs both tools into `~/miniforge3` and configures them for future Bash terminals.

Open a new remote terminal, then check `Conda`.

```
conda --version
```

Check `Mamba` separately.

```
mamba --version
```

Using Conda and Mamba

Once installed, create a separate environment for each project. Keep the initial `base` environment for managing the tools. The following examples use `Conda`. For these create, activate, install, and deactivate commands, you may replace `conda` with `mamba` to use `Mamba` instead.

Create an environment named `conda-demo` containing Python 3.11.

```
conda create -n conda-demo python=3.11
```

Review the proposed packages and enter `y` when prompted to continue. After creation finishes, activate the environment.

```
conda activate conda-demo
```

Activation selects this environment's Python and packages for the current terminal. Check its Python version.

```
python --version
```

Install NumPy version 1.26.4 into the active environment.

```
conda install numpy=1.26.4
```

Check the installed version.

```
python -c "import numpy; print(numpy.__version__)"
```

While the environment is active, run your programs with their normal Python commands. For example, if the current folder contains `hello.py`, run `python hello.py`.

When finished, leave the environment.

```
conda deactivate
```

In a new terminal, activate `conda-demo` again before returning to the project.

TROUBLESHOOTING & Additional Information

If login fails, contact your PI or project manager. To investigate the connection yourself, open Windows Terminal on your CESR computer.

Add `-vvv` before `-p` in your connection command. For example, replace the username below with your own CESR username and run the command for `COCESR-1141`.

```
ssh -vvv -p 222 -l jvaught@cesr.sc.edu 129.252.23.212
```

The `-vvv` option prints detailed diagnostic messages about the connection and authentication attempts. A timeout usually points to the address, port, or network access. A `Permission denied` message points to an authentication or account-access problem. Keep the exact error message so that you can search for it or provide it to support.

Server information

The two CESR workstation addresses are listed below.

Workstation	IP address	SSH port
`COCESR-1141`	`129.252.23.212`	222
`COCESR-1034`	`10.173.112.77`	222